

TUGAS INDIVIDU

**PENGEMBANGAN SISTEM NAVIGASI CERDAS
SIMULASI PATHFINDING PADA PETA GENERATIF**



2401020105 - MIKAEL REJEKI SITUMORANG

(c) 2026
Teknik Informatika
Jurusan Teknik Elektro dan Informatika
Fakultas Teknik dan Teknologi Kemaritiman
Universitas Maritim Raja Ali Haji

DAFTAR ISI

COVER	i
DAFTAR ISI	ii
DAFTAR TABEL	iii
DAFTAR GAMBAR	iv
BAB I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Batasan Masalah	4
BAB II LANDASAN TEORITIS	5
II.1 Pencarian Rute (Graph dan Pathfinding)	5
II.2 Algoritma A* (A Star)	7
II.3 Pendekatan Teknik Algoritma	10
II.4 Analisis Kompleksitas Algoritma	11
II.5 Peta Generatif dan Simulasi Pathfinding	13
BAB III PEMBAHASAN KONTRIBUSI.....	15
III.1 Kontribusi Implementasi Generasi Peta Prosedural	15
III.2 Analisis Kompleksitas.....	28
DAFTAR PUSTAKA	33

DAFTAR TABEL

Tabel 1	Fungsi heuristik pada algoritma A*	9
Tabel 2	Perbandingan kompleksitas algoritma pencarian jalur	12
Tabel 3	Analisis kompleksitas fungsi generate_map()	29
Tabel 4	Analisis kompleksitas fungsi _grow_corridor()	30
Tabel 5	Analisis kompleksitas fungsi _flood_fill() (BFS)	31
Tabel 6	Analisis kompleksitas fungsi _heal_dead_ends_v2()	31
Tabel 7	Ringkasan analisis Big-O seluruh kontribusi	32

DAFTAR GAMBAR

Gambar 1 Struktur class Grid dan Tile pada gen.py	16
Gambar 2 Kode fungsi _grow_corridor()	18
Gambar 3 Kode fungsi _neighbor_constraints() dan _get_valid_options()	20
Gambar 4 Kode fungsi _flood_fill()	22
Gambar 5 Kode fungsi _heal_dead_ends_v2()	24
Gambar 6 Kode fungsi generate_map()	27

BAB I

PENDAHULUAN

I.1 Latar Belakang

Perkembangan teknologi informasi dan sistem komputasi cerdas pada era modern telah mendorong lahirnya berbagai aplikasi navigasi digital yang mampu membantu manusia dalam menentukan rute perjalanan secara efisien dan akurat. Kemampuan sistem untuk menemukan jalur optimal dari satu titik ke titik lain di dalam suatu jaringan merupakan permasalahan fundamental dalam ilmu komputer yang dikenal sebagai shortest path problem atau masalah jalur terpendek.

Dalam konteks kehidupan sehari-hari, kebutuhan akan navigasi cerdas tidak hanya terbatas pada sistem transportasi berskala besar seperti Google Maps atau Waze, tetapi juga mencakup permasalahan pencarian lokasi pada skala lokal. Salah satu skenario yang relevan adalah pencarian lokasi pedagang atau unit usaha mikro yang tersebar di suatu kawasan, misalnya pencarian gerobak kopi terdekat dalam suatu lingkungan perkotaan. Kemampuan sistem untuk menghitung rute terpendek menuju tujuan tersebut merupakan implementasi nyata dari algoritma pencarian jalur.

Permasalahan pencarian jalur dalam ilmu komputer diselesaikan dengan merepresentasikan lingkungan sebagai sebuah graph yang terdiri dari simpul (node) dan sisi (edge). Berbagai algoritma telah dikembangkan untuk menyelesaikan masalah ini, antara lain Breadth-First Search (BFS), algoritma Dijkstra, serta algoritma A* (A-Star). Di antara berbagai algoritma tersebut, A* dikenal sebagai salah satu pendekatan yang paling efisien karena memadukan biaya aktual perjalanan dengan estimasi heuristik menuju tujuan, sehingga mempercepat proses pencarian tanpa mengorbankan optimalitas jalur yang dihasilkan.

Pada sisi lain, teknik procedural generation atau generasi prosedural memungkinkan sistem komputer untuk menghasilkan struktur lingkungan secara otomatis melalui algoritma tertentu. Dalam konteks simulasi navigasi, teknik ini memungkinkan peta jalan kota dibangun secara acak setiap kali sistem dijalankan, sehingga menghasilkan lingkungan pengujian yang lebih dinamis dan bervariasi. Integrasi antara peta generatif dengan algoritma pathfinding membentuk sebuah sistem simulasi navigasi yang tidak hanya fungsional tetapi juga mampu

mendemonstrasikan cara kerja algoritma pada kondisi lingkungan yang berbeda-beda.

Berdasarkan latar belakang tersebut, proyek ini mengembangkan sebuah sistem bernama Seruni Map, yaitu sistem simulasi navigasi cerdas berbasis algoritma A* yang mampu mencari jalur terpendek menuju gerobak kopi Seruni terdekat pada peta kota yang dihasilkan secara generatif. Implementasi final sistem ini dilengkapi dengan fitur pencarian aset terdekat secara otomatis menggunakan mekanisme Breadth-First Search untuk menentukan tujuan, yang kemudian diintegrasikan dengan algoritma A* untuk menentukan jalur optimalnya. Sistem diimplementasikan menggunakan bahasa pemrograman Python dengan pustaka Pygame untuk antarmuka grafis dan simulasi.

I.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana merancang dan mengimplementasikan sistem yang mampu menghasilkan peta kota secara otomatis menggunakan metode generative map berbasis grid dua dimensi?
2. Bagaimana mengimplementasikan algoritma A* untuk menemukan jalur terpendek antara titik awal dengan titik tujuan pada representasi graf jaringan jalan yang dihasilkan secara generatif?
3. Bagaimana merancang mekanisme pencarian gerobak kopi (nearest asset search) terdekat dari posisi pengguna sebagai fitur pengembangan lanjutan dari rancangan awal sistem?
4. Bagaimana memvisualisasikan proses pencarian jalur secara interaktif dan real-time sehingga pengguna dapat mengamati eksplorasi node dan pembentukan jalur optimal secara langsung?
5. Bagaimana menganalisis kompleksitas waktu dan ruang dari algoritma yang diimplementasikan dalam sistem navigasi berbasis peta generatif?

I.3 Batasan Masalah

Agar pembahasan dalam laporan ini lebih terfokus dan terarah, beberapa batasan masalah ditetapkan sebagai berikut:

1. Sistem menggunakan representasi peta dalam bentuk grid dua dimensi (2D) dengan ukuran sel yang seragam, tidak menggunakan representasi tiga dimensi.
2. Algoritma pencarian jalur yang diimplementasikan hanya menggunakan algoritma A*; tidak dibandingkan dengan algoritma Dijkstra atau BFS secara langsung dalam implementasi utama.
3. Sistem merupakan simulasi offline; tidak terhubung dengan internet, layanan peta daring, maupun data geospasial nyata.
4. Peta yang dihasilkan tidak menggunakan data GPS nyata maupun representasi peta kota sesungguhnya, melainkan dibangun sepenuhnya secara prosedural.
5. Fokus utama sistem adalah pada shortest path berdasarkan heuristik Manhattan Distance; tidak mempertimbangkan faktor lalu lintas, kondisi jalan, atau biaya nyata perjalanan.
6. Sistem tidak mendukung multi-agent pathfinding; hanya satu kendaraan yang bergerak pada satu waktu.
7. Implementasi dilakukan menggunakan bahasa pemrograman Python dengan pustaka Pygame; tidak menggunakan framework game atau simulasi lainnya.

BAB II

LANDASAN TEORITIS

II.1 Pencarian Rute (Graph dan Pathfinding)

Pencarian rute (route searching atau pathfinding) merupakan proses komputasi untuk menemukan jalur dari suatu titik awal menuju titik tujuan dalam sebuah lingkungan yang direpresentasikan secara matematis. Dalam ilmu komputer, permasalahan ini dimodelkan menggunakan struktur data graph.

II.1.1 Konsep Graph

Secara formal, sebuah graph G didefinisikan sebagai pasangan terurut $G = (V, E)$, di mana V adalah himpunan simpul (node atau vertex) yang merepresentasikan titik-titik lokasi, dan E adalah himpunan sisi (edge) yang merepresentasikan koneksi atau jalur yang menghubungkan dua simpul.

Dalam konteks navigasi, node merepresentasikan persimpangan jalan, lokasi tujuan, atau titik-titik kunci pada jaringan jalan. Sementara itu, edge merepresentasikan segmen jalan yang menghubungkan dua node, di mana setiap edge memiliki bobot (weight) yang mencerminkan biaya perjalanan, misalnya jarak atau waktu tempuh.

Graph dapat bersifat berarah (directed graph atau digraph) jika sisi hanya dapat dilalui dalam satu arah, atau tidak berarah (undirected graph) jika sisi dapat dilalui dari kedua arah. Dalam sistem navigasi pada jalan umum, umumnya digunakan graf tidak berarah atau digraph tergantung pada ada tidaknya aturan satu arah.

II.1.2 Grid Pathfinding

Grid pathfinding merupakan pendekatan khusus dalam pencarian jalur di mana lingkungan direpresentasikan sebagai sebuah kisi dua dimensi (grid). Setiap sel dalam grid berperan sebagai sebuah node, dan koneksi terbentuk antara sel yang berdekatan secara horizontal, vertikal, atau diagonal. Grid pathfinding banyak digunakan dalam simulasi karena kemudahannya dalam diimplementasikan dan kemampuannya merepresentasikan lingkungan nyata secara sederhana.

Pada sistem Seruni Map, peta kota direpresentasikan sebagai grid dua dimensi berukuran GCOLS x GROWS, di mana setiap sel dapat berupa jalan (road) atau area kosong (empty). Algoritma pathfinding hanya bergerak melalui sel-sel yang merupakan jalan, menjadikan grid pathfinding sebagai fondasi representasi lingkungan sistem.

II.1.3 Shortest Path Problem

Shortest path problem adalah permasalahan pencarian jalur dengan total bobot edge terkecil dari node sumber ke node tujuan. Terdapat beberapa varian, antara lain Single-Source Shortest Path (dari satu sumber ke semua node lain) dan Single-Pair Shortest Path (dari satu sumber ke satu tujuan). Algoritma yang umum digunakan meliputi Dijkstra (optimal, tanpa heuristik), Bellman-Ford (mendukung bobot negatif), serta A* (optimal dengan heuristik).

II.2 Algoritma A* (A Star)

Algoritma A* (A Star) pertama kali diperkenalkan oleh Hart, Nilsson, dan Raphael pada tahun 1968 sebagai algoritma pencarian jalur yang menggabungkan kelebihan dari algoritma Dijkstra (menjamin jalur terpendek) dan algoritma Greedy Best-First Search (efisien dalam eksplorasi node). Algoritma ini diklasifikasikan sebagai informed search algorithm karena menggunakan informasi heuristik dalam proses pencariannya.

II.2.1 Fungsi Evaluasi $f(n)$

Inti dari algoritma A* adalah fungsi evaluasi yang digunakan untuk menentukan prioritas eksplorasi node. Fungsi evaluasi tersebut dinyatakan sebagai:

$$f(n) = g(n) + h(n)$$

Keterangan:

- $f(n)$: Nilai total estimasi biaya jalur melalui node n dari node awal ke node tujuan. Node dengan nilai $f(n)$ terkecil diprioritaskan untuk dieksplorasi.
- $g(n)$: Biaya aktual perjalanan dari node awal (start node) ke node n . Nilai $g(n)$ merepresentasikan jarak atau biaya nyata yang telah ditempuh.
- $h(n)$: Nilai heuristik, yaitu estimasi biaya dari node n menuju node tujuan (goal node). Nilai $h(n)$ harus bersifat admissible, artinya tidak pernah melebihi biaya aktual yang sesungguhnya agar A* menjamin optimalitas.

II.2.2 Fungsi Heuristik

Pemilihan fungsi heuristik $h(n)$ sangat menentukan efisiensi algoritma A*. Beberapa fungsi heuristik yang umum digunakan dalam grid pathfinding adalah:

Fungsi Heuristik	Rumus	Cocok Untuk
Manhattan Distance	$ x1-x2 + y1-y2 $	Grid 4-arah (atas, bawah, kiri, kanan)
Euclidean Distance	$\text{sqrt}((x1-x2)^2 + (y1-y2)^2)$	Pergerakan 8-arah atau bebas
Chebyshev Distance	$\max(x1-x2 , y1-y2)$	Grid 8-arah dengan biaya diagonal sama

Tabel 1. Fungsi Heuristik pada Algoritma A*

II.2.3 Open List dan Closed List

Algoritma A* mengelola dua himpunan node selama proses pencariannya. Open List adalah himpunan node yang telah ditemukan tetapi belum dievaluasi sepenuhnya. Node dalam open list diurutkan berdasarkan nilai $f(n)$, sehingga node dengan $f(n)$ terkecil selalu dievaluasi pertama. Struktur data yang efisien untuk implementasi open list adalah priority queue atau min-heap, yang memungkinkan operasi pengambilan elemen minimum dalam $O(\log n)$. Closed List adalah himpunan node yang telah dievaluasi sepenuhnya dan tidak akan dievaluasi ulang.

II.3 Pendekatan Teknik Algoritma

Algoritma yang digunakan dalam proyek ini dapat dipahami melalui tiga pendekatan teknik yang mendasarinya.

II.3.1 Greedy Approach

Greedy approach adalah strategi pengambilan keputusan yang pada setiap langkah selalu memilih opsi yang tampak paling baik pada saat itu (locally optimal), dengan harapan bahwa pilihan lokal yang optimal akan menghasilkan solusi global yang optimal. Dalam generasi peta prosedural, greedy approach diterapkan pada pemilihan tipe tile terbaik yang memenuhi constraint dari tetangga, dengan preferensi pada tile yang paling sederhana (jumlah port minimum).

II.3.2 Heuristic Search

Heuristic search adalah teknik pencarian yang memanfaatkan fungsi heuristik sebagai panduan untuk mengevaluasi seberapa menjanjikan suatu node dalam proses pencarian solusi. Pada algoritma A* yang digunakan sistem ini, fungsi heuristik Manhattan Distance memberikan estimasi biaya dari node saat ini ke tujuan. Kualitas heuristik yang admissible dan consistent menjamin A* selalu menemukan jalur optimal.

II.3.3 Informed Search Algorithm

Informed search atau knowledge-based search adalah kategori algoritma pencarian yang menggunakan informasi tambahan tentang domain permasalahan untuk memandu proses pencarian. A* merupakan contoh klasik informed search algorithm yang menjamin optimalitas (completeness dan optimality) jika fungsi heuristik yang digunakan bersifat admissible (tidak melebihi-lebihkan biaya sebenarnya) dan consistent (memenuhi ketidaksamaan segitiga).

II.4 Analisis Kompleksitas Algoritma

Analisis kompleksitas merupakan kajian teoritis untuk mengukur efisiensi suatu algoritma ditinjau dari penggunaan sumber daya komputasi, yaitu waktu (time complexity) dan memori (space complexity).

II.4.1 Time Complexity

Kompleksitas waktu algoritma A* dalam kasus terburuk (worst case) adalah $O(b^d)$, di mana b adalah branching factor (jumlah rata-rata tetangga per node) dan d adalah kedalaman solusi (panjang jalur terpendek). Namun, dengan heuristik yang baik, A* dapat berperforma jauh lebih baik. Pada implementasi dengan priority queue berbasis binary heap, kompleksitas total A* dengan n node yang dieksplorasi adalah $O(n \log n)$, atau $O(|V| \log |V|)$ di mana $|V|$ adalah jumlah node yang dieksplorasi.

II.4.2 Space Complexity

Kompleksitas ruang algoritma A* adalah $O(b^d)$ karena dalam kasus terburuk, semua node yang dihasilkan harus disimpan dalam memori. Pada

implementasi praktis untuk grid pathfinding, penggunaan dictionary untuk menyimpan g-score dan came_from menghasilkan kompleksitas ruang $O(|V|)$ di mana $|V|$ adalah jumlah total node yang pernah dimasukkan ke dalam open list.

II.4.3 Perbandingan Efisiensi

Algoritma	Time Complexity	Space Complexity	Optimalitas
BFS	$O(V + E)$	$O(V)$	Ya (bobot seragam)
Dijkstra	$O((V+E) \log V)$	$O(V)$	Ya
Greedy BFS	$O(b^d)$	$O(b^d)$	Tidak
A*	$O(b^d) / O(n \log n)$	$O(b^d)$	Ya (h admissible)

Tabel 2. Perbandingan Kompleksitas Algoritma Pencarian Jalur

II.5 Peta Generatif dan Simulasi Pathfinding

Peta generatif (generative map) adalah teknik pembuatan lingkungan secara prosedural menggunakan algoritma komputasi, tanpa intervensi manual dari perancang. Teknik ini merupakan bagian dari konsep yang lebih luas yang disebut Procedural Content Generation (PCG), yang banyak diaplikasikan dalam pengembangan game, simulasi kota virtual, dan penelitian kecerdasan buatan.

Pada sistem Seruni Map, peta kota dihasilkan secara prosedural menggunakan algoritma Corridor Snake Growth yang membangun jaringan jalan dalam bentuk grid dua dimensi. Setiap sel pada grid diklasifikasikan menjadi beberapa tipe: jalan lurus (straight), tikungan (curve), persimpangan-T (T-junction), dan persimpangan silang (cross). Setiap kali sistem diinisialisasi dengan nilai seed berbeda, struktur peta yang dihasilkan pun berbeda, memberikan lingkungan pengujian yang dinamis dan bervariasi.

Simulasi pathfinding merupakan visualisasi interaktif dari proses pencarian jalur yang memungkinkan pengguna mengamati setiap tahap eksplorasi node dan pembentukan jalur secara real-time. Dalam simulasi ini, node-node yang telah dieksplorasi oleh algoritma ditampilkan dengan warna berbeda, jalur optimal divisualisasikan sebagai garis pada peta, dan sebuah kendaraan virtual bergerak menyusuri jalur tersebut untuk memberikan representasi yang intuitif dari hasil navigasi.

BAB III

PEMBAHASAN KONTRIBUSI

Bab ini membahas kontribusi teknis yang telah dikembangkan dalam proyek Seruni Map. Kontribusi utama penulis berfokus pada perancangan dan implementasi Algoritma Generasi Peta Prosedural yang terdapat pada modul `gen.py`. Modul ini merupakan fondasi dari seluruh sistem karena tanpa peta yang valid dan terkoneksi, algoritma pathfinding A* tidak dapat bekerja dengan baik. Kode sumber lengkap dapat diakses melalui repositori GitHub: <https://github.com/SpyHolder/Project-GRAFKOM>

III.1 Kontribusi Implementasi Generasi Peta Prosedural

Lingkungan simulasi pada proyek ini tidak dibangun secara manual menggunakan matriks grid statis, melainkan digenerasi secara prosedural setiap kali program dijalankan. Untuk merealisasikan hal ini, diterapkan algoritma Corridor Snake Growth yang menghasilkan jaringan jalan organik dan terkoneksi penuh. Implementasi mencakup enam komponen utama yang saling berintegrasi.

III.1.1 Struktur Data Grid dan Tile

Fondasi dari seluruh sistem generasi peta adalah dua class utama, yaitu `Tile` dan `Grid`. Class `Tile` menyimpan dua atribut: tipe tile (`EMPTY`, `STRAIGHT`, `CURVE`, `TJUNCTION`, `CROSS`) dan rotasinya. Class `Grid` merupakan matriks dua dimensi berukuran `cols` x `rows` yang menampung semua objek `Tile`, dilengkapi method `is_road()` untuk memeriksa apakah sebuah sel merupakan jalan.

Gambar 1. Struktur Class Grid dan Tile pada gen.py

```
1 class Tile:
2     def __init__(self):
3         self.type = EMPTY
4         self.rotation = 0
5
6
7 class Grid:
8     def __init__(self, cols, rows):
9         self.cols = cols
10        self.rows = rows
11        self.cells = [[Tile() for _ in range(cols)] for _ in range(rows)]
12
13    def get(self, col, row):
14        if 0 <= col < self.cols and 0 <= row < self.rows:
15            return self.cells[row][col]
16        return None
17
18    def set_tile(self, col, row, tile_type, rotation=0):
19        self.cells[row][col].type = tile_type
20        self.cells[row][col].rotation = rotation
21
22    def is_road(self, col, row):
23        t = self.get(col, row)
24        return t is not None and t.type != EMPTY
25
```

III.1.2 Algoritma Pertumbuhan Koridor (Corridor Snake Growth)

Fungsi `_grow_corridor()` adalah mesin utama pembentukan jalan. Algoritma ini menyimulasikan seekor 'ular' yang berjalan di grid, meninggalkan jejak berupa tile jalan. Setiap koridor memiliki arah awal, dan berbelok secara acak setiap 2-4 langkah untuk menghasilkan jalan yang organik dan tidak monoton.

Mekanisme kunci algoritma ini adalah `turn_after` — sebuah variabel acak yang menentukan setiap berapa langkah lurus koridor akan berbelok. Ketika giliran berbelok tiba, koridor mencoba membelok ke kiri atau kanan. Jika pembelokan tidak memungkinkan (keluar batas atau menghasilkan persimpangan berlebih), koridor tetap lurus. Sistem anti-cluster juga diterapkan: jika sebuah posisi sudah memiliki persimpangan di dekatnya, tile yang akan ditempatkan di-downgrade menjadi STRAIGHT atau CURVE.

Gambar 2. Kode Fungsi `_grow_corridor()`

```
1 def _grow_corridor(grid, start_c, start_r, start_dir, rng, max_len=None, cols=11, rows=11):
2     """
3     Grow a winding corridor from (start_c, start_r) in direction start_dir.
4     Returns number of tiles placed.
5     The corridor turns every 2-5 tiles using CURVE tiles for organic paths.
6     """
7     if max_len is None:
8         max_len = rng.randint(4, max(5, (cols + rows) // 3))
9
10    c, r, d = start_c, start_r, start_dir
11    placed = 0
12    straight_count = 0
13    turn_after = rng.randint(2, 4) # turn after this many straight tiles
14
15    for step in range(max_len):
16        dc, dr = DIR_DELTA[d]
17        nc, nr = c + dc, r + dr
18
19        # Bounds check
20        if not (0 <= nc < cols and 0 <= nr < rows):
21            break
22
23        # Hit existing road - try to connect and stop
24        if grid.is_road(nc, nr):
25            _upgrade_tile_port(grid, c, r, d)
26            _upgrade_tile_port(grid, nc, nr, OPPOSITE[d])
27            break
28
29        # Decide: continue straight or turn?
30        incoming = OPPOSITE[d]
31        outgoing = d # default: straight
32
33        if straight_count >= turn_after:
34            # Time to turn - pick left or right
35            turn_dir = (d + 1) % 4 if rng.random() < 0.5 else (d + 3) % 4
36            # Check if turn is possible (not out of bounds in 2 steps)
37            tdc, tdr = DIR_DELTA[turn_dir]
38            future_c, future_r = nc + tdc, nr + tdr
39            if 0 <= future_c < cols and 0 <= future_r < rows:
40                outgoing = turn_dir
41                straight_count = 0
42                turn_after = rng.randint(2, 4)
43
44        # Find appropriate tile
45        tile_info = _find_tile_for_ports(grid, nc, nr, incoming, outgoing)
46        if tile_info is None:
47            # Try just incoming (straight continuation)
48            tile_info = _find_tile_for_ports(grid, nc, nr, incoming, d)
49            outgoing = d
50        if tile_info is None:
51            break
52
53        # Anti-cluster: reject if this would be an intersection next to another
54        if tile_info[0] in (TJUNCTION, CROSS):
55            if _has_adjacent_intersection(grid, nc, nr):
56                # Try simpler tile instead
57                simple = [(t, rot) for t, rot in _get_options_with_ports(grid, nc, nr, {incoming})
58                        if t in (STRAIGHT, CURVE)]
59                if simple:
60                    tile_info = simple[0]
61                    outgoing = d # just go straight
62                else:
63                    break
64
65        # Place tile
66        grid.set_tile(nc, nr, tile_info[0], tile_info[1])
67        placed += 1
68
69        # Ensure previous tile connects forward
70        if grid.is_road(c, r):
71            _upgrade_tile_port(grid, c, r, d)
72
73        # Advance
74        c, r = nc, nr
75        actual_ports = get_ports(tile_info[0], tile_info[1])
76        if outgoing in actual_ports:
77            d = outgoing
78            if outgoing == start_dir or tile_info[0] == STRAIGHT:
79                straight_count += 1
80            else:
81                straight_count = 0
82        else:
83            # Tile doesn't have outgoing port, stop
84            break
85
86    return placed
```

III.1.3 Constraint Satisfaction Problem (CSP) — Penyelarasan Tile

Setiap tile yang ditempatkan harus memenuhi constraint dari tetangganya: jika tile A memiliki port yang mengarah ke tile B, maka tile B harus memiliki port yang mengarah balik ke A. Masalah ini dimodelkan sebagai Constraint Satisfaction Problem (CSP).

Fungsi `_neighbor_constraints()` mengumpulkan semua constraint dari tile-tile tetangga yang sudah ada. Fungsi `_get_valid_options()` kemudian mengevaluasi semua kombinasi tipe dan rotasi tile untuk menemukan yang memenuhi semua constraint tersebut. Fungsi `_find_tile_for_ports()` memilih opsi paling sederhana (jumlah port minimum) yang memiliki kedua port yang dibutuhkan (incoming dan outgoing).

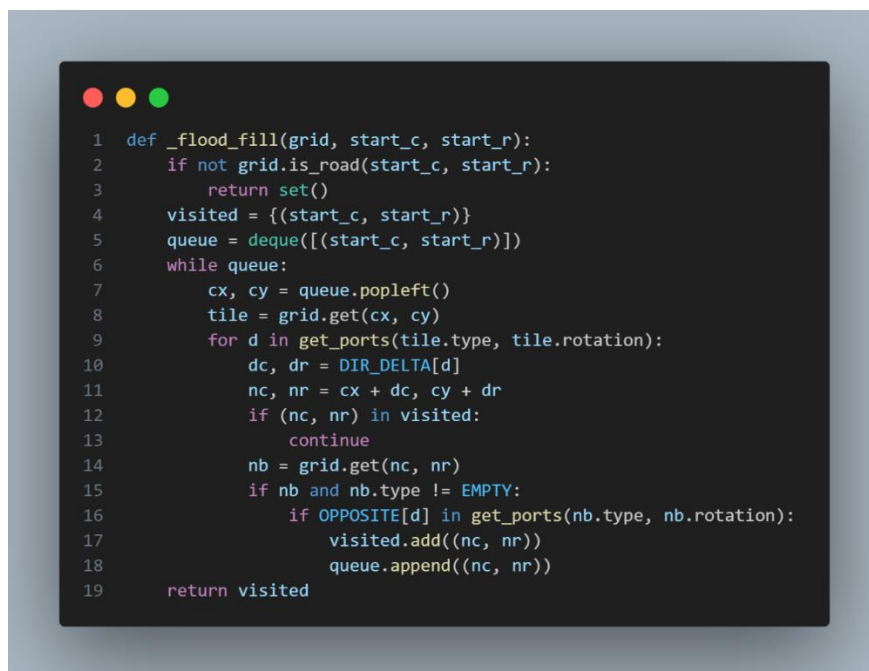
Gambar 3. Kode Fungsi `_neighbor_constraints()` dan `_get_valid_options()`

```
1 def _neighbor_constraints(grid, c, r):
2     constraints = {}
3     for d, (dc, dr) in DIR_DELTA.items():
4         nb = grid.get(c + dc, r + dr)
5         if nb is None or nb.type == EMPTY:
6             continue
7         opp = OPPOSITE[d]
8         nb_ports = get_ports(nb.type, nb.rotation)
9         constraints[d] = opp in nb_ports
10    return constraints
11
12
13 def _get_valid_options(grid, c, r):
14     constraints = _neighbor_constraints(grid, c, r)
15     valid = []
16     for tile_type, rotations in TILE_PORTS.items():
17         for rot, ports in enumerate(rotations):
18             ok = True
19             for d, must_open in constraints.items():
20                 if (d in ports) != must_open:
21                     ok = False
22                     break
23             if ok:
24                 valid.append((tile_type, rot))
25    return valid
```


III.1.4 Verifikasi Konektivitas — BFS Flood Fill

Algoritma pencarian jalur A* memerlukan konektivitas graph jaringan jalan yang sempurna. Untuk memverifikasi hal ini, diterapkan algoritma Breadth-First Search (BFS) dalam bentuk flood fill pada fungsi `_flood_fill()`. Fungsi ini memulai dari satu tile jalan dan menelusuri semua tile yang dapat dijangkau melalui port yang saling terhubung. Jika ada tile jalan yang tidak dapat dijangkau (terisolasi), tile tersebut merupakan komponen terpisah yang perlu disambungkan atau dihapus.

Gambar 4. Kode Fungsi `_flood_fill()`

A screenshot of a code editor with a dark background and light-colored text. The code is written in Python and implements a Breadth-First Search (BFS) flood fill algorithm. It starts by checking if the start coordinates are a road. If not, it returns an empty set. If yes, it initializes a visited set and a queue. It then enters a while loop that processes the queue, checking neighbors and adding them to the queue if they are not visited and are roads. The function returns the visited set.

```
1 def _flood_fill(grid, start_c, start_r):
2     if not grid.is_road(start_c, start_r):
3         return set()
4     visited = {(start_c, start_r)}
5     queue = deque([(start_c, start_r)])
6     while queue:
7         cx, cy = queue.popleft()
8         tile = grid.get(cx, cy)
9         for d in get_ports(tile.type, tile.rotation):
10             dc, dr = DIR_DELTA[d]
11             nc, nr = cx + dc, cy + dr
12             if (nc, nr) in visited:
13                 continue
14             nb = grid.get(nc, nr)
15             if nb and nb.type != EMPTY:
16                 if OPPOSITE[d] in get_ports(nb.type, nb.rotation):
17                     visited.add((nc, nr))
18                     queue.append((nc, nr))
19     return visited
```

III.1.5 Penyembuhan Jalan Buntu (Dead-End Healing)

Setelah semua koridor tumbuh, bisa terdapat tile dengan hanya satu koneksi aktif (dead-end). Tile semacam ini tidak ideal karena membatasi kemampuan navigasi A*. Fungsi `_heal_dead_ends_v2()` menangani kondisi ini dengan empat strategi bertahap yang dieksekusi secara iteratif hingga tidak ada dead-end interior yang tersisa.

Strategi 1 adalah memperpanjang jalan ke arah batas peta menggunakan BFS untuk menemukan jalur menuju border. Strategi 2 adalah membuat loop ke jalan terdekat dalam radius tertentu. Strategi 3 adalah mengupgrade tipe tile agar mendapatkan lebih banyak koneksi. Jika semua strategi gagal, Strategi 4 menghapus tile tersebut dari grid (set ke EMPTY)

Gambar 5. Kode Fungsi `_heal_dead_ends_v2()` — Strategi Penyembuhan

```
1 def _heal_dead_ends_v2(grid, seed_c, seed_r, rng):
2     cols, rows = grid.cols, grid.rows
3     for _pass in range(15):
4         dead = _find_dead_ends(grid)
5         interior_dead = [(c, r) for c, r in dead
6                         if not _is_border(c, r, cols, rows)]
7         if not interior_dead:
8             break
9         progress = False
10        for c, r in interior_dead:
11            if not grid.is_road(c, r) or _tile_degree(grid, c, r) >= 2:
12                continue
13            tile = grid.get(c, r)
14            ports = _get_ports(tile.type, tile.rotation)
15            unconnected = []
16            for d in ports:
17                dc, dr = DIR_DELTA[d]
18                nb = grid.get(c + dc, r + dr)
19                if nb is None:
20                    continue
21                if nb.type == EMPTY:
22                    unconnected.append(d)
23                elif OPPOSITE[d] not in _get_ports(nb.type, nb.rotation):
24                    unconnected.append(d)
25            free_dirs = [d for d in range(4) if d not in ports]
26            healed = False
27
28            # Strategy 1: extend toward border
29            for d in unconnected + free_dirs:
30                dc, dr = DIR_DELTA[d]
31                nc, nr = c + dc, r + dr
32                nb = grid.get(nc, nr)
33                if nb is None or nb.type != EMPTY:
34                    continue
35                border_targets = [(bc, br) for br in range(rows)
36                                for bc in range(cols)
37                                if _is_border(bc, br, cols, rows)
38                                and not grid.is_road(bc, br)]
39                if not border_targets:
40                    continue
41                path = _bfs_path_to_target(grid, nc, nr, border_targets)
42                if path and len(path) <= max(8, min(cols, rows) // 4):
43                    if d not in ports:
44                        _upgrade_tile_port(grid, c, r, d)
45                    if _build_road_along_path(grid, [(c, r)] + path, rng) > 0:
46                        healed = progress = True
47                        break
48            if healed:
49                continue
50
51            # Strategy 2: loop to nearby road
52            for d in unconnected + free_dirs:
53                dc, dr = DIR_DELTA[d]
54                nc, nr = c + dc, r + dr
55                nb = grid.get(nc, nr)
56                if nb is None or nb.type != EMPTY:
57                    continue
58                search_r = max(4, min(cols, rows) // 8)
59                nearby = [(cc, rr)
60                        for rr in range(max(0, r-search_r), min(rows, r+search_r+1))
61                        for cc in range(max(0, c-search_r), min(cols, c+search_r+1))
62                        if (cc, rr) != (c, r) and grid.is_road(cc, rr)]
63                path = _bfs_path_to_target(grid, nc, nr, nearby)
64                if path and len(path) <= max(5, min(cols, rows) // 6):
65                    if d not in ports:
66                        _upgrade_tile_port(grid, c, r, d)
67                    if _build_road_along_path(grid, [(c, r)] + path, rng) > 0:
68                        healed = progress = True
69                        break
70            if healed:
71                continue
72
73            # Strategy 3: upgrade tile
74            opts = _get_valid_options(grid, c, r)
75            opts.sort(key=lambda o: _tile_degree_if_placed(
76                    grid, c, r, o[0], o[1]), reverse=True)
77            if opts and _tile_degree_if_placed(grid, c, r, opts[0][0], opts[0][1]) >= 2:
78                grid.set_tile(c, r, opts[0][0], opts[0][1])
79                progress = True
80                continue
81
82            # Strategy 4: remove
83            grid.set_tile(c, r, EMPTY, 0)
84            progress = True
85
86        if not progress:
87            break
```

III.1.6 Fungsi Utama `generate_map()`

Fungsi `generate_map()` adalah orkestrator dari seluruh proses generasi peta. Fungsi ini menerima parameter `cols`, `rows`, dan `seed`, kemudian menjalankan lima fase secara berurutan: (1) penempatan tile awal (seed point) di tengah grid, (2) pertumbuhan koridor menggunakan tips-driven expansion, (3) pemaksaan border exits untuk memastikan ada jalan ke tepi peta, (4) dead-end healing, dan (5) cleanup serta verifikasi komponen terhubung terbesar.

Gambar 6. Kode Fungsi `generate_map()`

```
# gen.py - Fungsi Utama Generasi Peta
def generate_map(cols, rows, seed=None):
    """
    Orkestrasi 5 fase generasi peta prosedural:
    Phase 1: Seed tile di tengah grid
    Phase 2: Pertumbuhan koridor berbasis tips
    Phase 3: Force border exits
    Phase 4: Dead-end healing
    Phase 5: Cleanup + ambil komponen terbesar
    """
    rng = random.Random(seed)      # seed deterministik: peta sama untuk seed
    sama
    grid = Grid(cols, rows)
    seed_c, seed_r = cols // 2, rows // 2

    # Phase 1: Letakkan STRAIGHT tile di tengah sebagai titik awal
    start_dir = rng.choice([0, 1, 2, 3])
    grid.set_tile(seed_c, seed_r, STRAIGHT, start_dir % 2)

    target_exits = _choose_border_exits(cols, rows, rng)
    max_roads = int(cols * rows * 0.35)    # maks 35% grid adalah jalan

    # Phase 2: Tumbuhkan koridor dari tips (ujung-ujung jalan yang bisa
    berkembang)
    seed_ports = list(get_ports(STRAIGHT, start_dir % 2))
    tips = [(seed_c, seed_r, d) for d in seed_ports]
    road_count = 1
    max_iterations = cols * rows * 2
    iteration = 0

    while tips and road_count < max_roads and iteration < max_iterations:
        iteration += 1
        idx = rng.randint(0, len(tips) - 1)
        tc, tr, td = tips.pop(idx)
```

```

if not grid.is_road(tc, tr): continue

corridor_len = rng.randint(3, max(4, min(cols, rows) // 2))
placed = _grow_corridor(grid, tc, tr, td, rng,
                        max_len=corridor_len, cols=cols, rows=rows)
road_count = len(_all_road_cells(grid))

# Dari koridor baru, tambahkan tips untuk percabangan (prob 18%)
if placed > 0:
    c, r, d = tc, tr, td
    for step in range(placed + 1):
        dc, dr = DIR_DELTA[d]
        nc, nr = c + dc, r + dr
        if not grid.is_road(nc, nr): break
        if step > 1 and rng.random() < 0.18 and road_count < max_roads:
            branch_dirs = [(d+1)%4, (d+3)%4]
            rng.shuffle(branch_dirs)
            for bd in branch_dirs:
                if _count_nearby_intersections(grid, nc, nr, 2) >= 1:
                    continue
                bdc, bdr = DIR_DELTA[bd]
                bnb = grid.get(nc + bdc, nr + bdr)
                if bnb and bnb.type == EMPTY:
                    if _upgrade_tile_port(grid, nc, nr, bd):
                        tips.append((nc, nr, bd))
                    break

# Phase 3: Paksa border exits agar ada jalan ke tepi peta
for ec, er in target_exits:
    if grid.is_road(ec, er): continue
    road_cells = _all_road_cells(grid)
    path = _bfs_path_to_target(grid, ec, er, road_cells)
    if path:
        _build_road_along_path(grid, list(reversed(path)), rng)

# Phase 4: Sembuhkan dead-end interior
_heal_dead_ends_v2(grid, seed_c, seed_r, rng)

# Phase 5: Hapus isolated tiles, ambil komponen terhubung terbesar
all_roads = _all_road_cells(grid)
if all_roads:
    remaining = set(all_roads)
    components = []
    while remaining:
        start = next(iter(remaining))
        comp = _flood_fill(grid, start[0], start[1])

```

```

        components.append(comp)
        remaining -= comp
    # Coba hubungkan komponen kecil ke komponen terbesar
    if len(components) > 1:
        components.sort(key=len, reverse=True)
        main = components[0]
        for comp in components[1:]:
            best_dist = 9999
            best_a = best_b = None
            for a in comp:
                for b in main:
                    dd = abs(a[0]-b[0]) + abs(a[1]-b[1])
                    if dd < best_dist:
                        best_dist = dd; best_a, best_b = a, b
            if best_a and best_b and best_dist <= 6:
                path = _bfs_path_to_target(
                    grid, best_a[0], best_a[1], {best_b})
                if path:
                    _build_road_along_path(grid, path, rng)
                    main = main | comp; continue
        for cx, cy in comp:
            grid.set_tile(cx, cy, EMPTY, 0) # hapus komponen
    terisolasi

    # Final healing dan downgrade tile yang berlebihan
    _heal_dead_ends_v2(grid, seed_c, seed_r, rng)
    _heal_dead_ends_v2(grid, seed_c, seed_r, rng)

    return grid

```

III.2 Analisis Kompleksitas

Bagian ini menganalisis kompleksitas waktu dan ruang dari setiap fungsi yang dikontribusikan ke dalam proyek Seruni Map menggunakan notasi Big-O. Analisis dilakukan berdasarkan parameter utama: N = jumlah total sel grid (cols x rows), V = jumlah sel jalan (road tiles), E = jumlah edge antar sel jalan, dan $|path|$ = panjang jalur yang dihasilkan.

III.2.1 Kompleksitas `generate_map()`

Fungsi `generate_map()` mengorkestrasi lima fase. Fase 1 berupa penempatan satu tile sehingga $O(1)$. Fase 2 adalah pertumbuhan koridor: setiap iterasi memanggil `_grow_corridor()` yang berjalan $O(L)$ di mana L adalah panjang koridor. Jumlah iterasi dibatasi oleh `max_iterations = cols x rows x 2`, sehingga fase

ini memiliki kompleksitas $O(N)$. Fase 3 (border exits) menjalankan BFS untuk setiap exit dengan kompleksitas $O(N)$ per exit, dan jumlah exit dibatasi konstan sehingga total $O(N)$. Fase 4 memanggil `_heal_dead_ends_v2()` dengan kompleksitas $O(V \times \text{pass})$. Fase 5 menjalankan `_flood_fill()` berulang kali dengan total $O(V + E)$.

Fase	Time Complexity	Keterangan
Phase 1 (Seed)	$O(1)$	Satu tile ditempatkan di tengah
Phase 2 (Corridor Growth)	$O(N)$	$\text{max_iterations} = \text{cols} \times \text{rows} \times 2$
Phase 3 (Border Exits)	$O(N)$	BFS per exit, jumlah exit konstan
Phase 4 (Healing)	$O(V \times \text{pass})$	Maks 15 iterasi per pemanggilan, 3x dipanggil
Phase 5 (Cleanup)	$O(V + E)$	Flood fill seluruh komponen jalan
Total <code>generate_map()</code>	$O(N)$	$N = \text{cols} \times \text{rows}$ (total sel grid)

Tabel 3. Analisis Kompleksitas Fungsi `generate_map()`

III.2.2 Kompleksitas `_grow_corridor()`

Fungsi `_grow_corridor()` menjalankan loop sebanyak maksimal `max_len` kali. Di setiap iterasi, operasi utama adalah `_find_tile_for_ports()` yang mengevaluasi semua kombinasi tile (jumlah kombinasi konstan, dibatasi oleh `TILE_PORTS`), sehingga $O(1)$ per iterasi. Total kompleksitas fungsi ini adalah $O(L)$ di mana L adalah panjang koridor.

Operasi	Time Complexity	Keterangan
Loop utama (<code>max_len</code> iterasi)	$O(L)$	L = panjang koridor (dibatasi)
<code>_find_tile_for_ports()</code> per iterasi	$O(1)$	Jumlah kombinasi tile konstan
<code>_has_adjacent_intersection()</code>	$O(1)$	Memeriksa 4 tetangga terdekat saja

Total _grow_corridor()	$O(L)$	Linier terhadap panjang koridor
------------------------	--------	---------------------------------

Tabel 4. Analisis Kompleksitas Fungsi _grow_corridor()

III.2.3 Kompleksitas _flood_fill() — BFS Konektivitas

Fungsi _flood_fill() mengimplementasikan BFS standar pada graph tile jalan. Setiap tile jalan (V) dikunjungi tepat satu kali, dan setiap edge (E) diperiksa dua kali (dari kedua endpoint). Operasi set membership (visited) menggunakan Python set sehingga $O(1)$ rata-rata. Menggunakan deque dari collections untuk queue memastikan operasi popleft() dalam $O(1)$.

Operasi	Complexity	Keterangan
Kunjungan node (tile jalan)	$O(V)$	Setiap tile dikunjungi tepat 1x
Pemeriksaan edge	$O(E)$	$E \approx 4V$ untuk grid 4-arrah
Set membership check	$O(1)$ avg	Python set hash-based lookup
Total _flood_fill()	$O(V + E)$	BFS klasik pada graph jalan

Tabel 5. Analisis Kompleksitas Fungsi _flood_fill() (BFS)

III.2.4 Kompleksitas _heal_dead_ends_v2()

Fungsi _heal_dead_ends_v2() menjalankan hingga 15 iterasi pass. Di setiap pass, _find_dead_ends() menelusuri seluruh grid $O(N)$. Untuk setiap dead-end yang ditemukan, strategi 1 dan 2 masing-masing menjalankan BFS $O(N)$. Dalam skenario terburuk dengan banyak dead-end, kompleksitasnya menjadi $O(\text{pass} \times V \times N)$. Namun dalam praktik, jumlah dead-end interior sedikit dan BFS berhenti lebih awal, sehingga performanya jauh lebih baik dari kasus terburuk.

Operasi	Worst Case	Keterangan
_find_dead_ends() per pass	$O(N)$	Traversal seluruh grid
BFS per dead-end (strategi 1&2)	$O(N)$	Mencari jalur ke border/road terdekat
Total (worst case)	$O(\text{pass} \times D \times N)$	D = jumlah dead-end, dibatasi 15 pass

Total (praktik rata-rata)	$O(V)$	Dead-end sedikit, BFS early-stop
---------------------------	--------	----------------------------------

Tabel 6. Analisis Kompleksitas Fungsi `_heal_dead_ends_v2()`

III.2.5 Ringkasan Analisis Big-O

Fungsi / Komponen	Time Complexity	Space Complexity	Keterangan
<code>generate_map()</code>	$O(N)$	$O(N)$	$N = \text{cols} \times \text{rows}$
<code>_grow_corridor()</code>	$O(L)$	$O(1)$	$L = \text{panjang koridor}$
<code>_find_tile_for_ports()</code> (CSP)	$O(1)$	$O(1)$	Kombinasi tile konstan
<code>_flood_fill()</code> (BFS)	$O(V + E)$	$O(V)$	$V = \text{tile jalan}$, $E = \text{koneksi}$
<code>_heal_dead_ends_v2()</code>	$O(V)$ avg	$O(V)$	Praktik jauh lebih baik
<code>_neighbor_constraints()</code> (CSP)	$O(1)$	$O(1)$	Maks 4 tetangga diperiksa
<code>_get_valid_options()</code> (CSP)	$O(1)$	$O(1)$	Jumlah tile/rotasi konstan

Tabel 7. Ringkasan Analisis Big-O Seluruh Kontribusi

Dari analisis di atas, terlihat bahwa keseluruhan sistem generasi peta memiliki kompleksitas dominan $O(N)$ di mana N adalah total sel grid. Ini merupakan kompleksitas yang sangat efisien mengingat setiap sel pada grid perlu setidaknya diperiksa satu kali untuk menentukan apakah akan menjadi jalan atau tidak. Tidak ada komponen yang memiliki kompleksitas superlinear terhadap N , memastikan sistem tetap responsif bahkan untuk peta berukuran besar.

DAFTAR PUSTAKA

- Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson Education.
- Patel, A. (2021). Introduction to A*. Red Blob Games. Diakses dari <https://www.redblobgames.com/pathfinding/a-star/introduction.html>
- Patel, A. (2021). Implementation of A*. Red Blob Games. Diakses dari <https://www.redblobgames.com/pathfinding/a-star/implementation.html>
- Shaker, N., Togelius, J., & Nelson, M. J. (2016). Procedural Content Generation in Games. Springer. Diakses dari <http://pcgbook.com>
- Skiena, S. S. (2020). The Algorithm Design Manual (3rd ed.). Springer. <https://doi.org/10.1007/978-3-030-54256-6>
- Pygame Community. (2023). Pygame Documentation. Diakses dari <https://www.pygame.org/docs/>
- Python Software Foundation. (2023). heapq - Heap queue algorithm. Python 3 Documentation. Diakses dari <https://docs.python.org/3/library/heapq.html>
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.